Real README

03 November 2026 18:45

The learning has been done with the help of famous Andrej Karpathey video: let's make GPT from scratch. The project was inspired by one of my previous project's, the TVC (thrust vector control) rocket GIMBAL making. I have completely made a decoder only transformer from scratch, by training it on the pure Pytorch.I wanted to somehow connect the complex Kalman Filter and the PID Controller with the AI, but the AI should be made by me (from TVC project).I have so far kept the track of my whole decoder transformer making journey with making personal notes, which can act as an open source learning material for beginners like me when I did it at the first place.

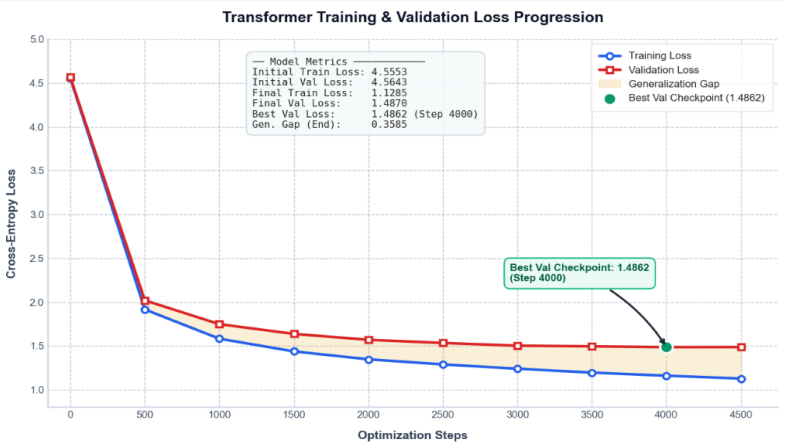
There have been total division of the complex topics like SELF attention, MULTI-HEAD attention, Feedforward block, decoder block, code in a linear and easy flow. The learning from mistake and correctly understanding the depth of the concept was done with the help of antigravity. Similarly, many similar text from it can be seen in the resources. The notes includes understanding each line of code and why we have taken only that way, or included only that pattern or the method is explained, code implemented, explaining each concept with real life and easy understanding examples, proper flow of diagram, important points have been marked as (IMP), etc has been the main part of the notes.

I see this as a beginner guide developing transformer from scratch all by myself, which will surely help you to built one for yourself too.

Project duration : Time needed to built the decoder transformer (GPT) : less than 9 day's.

I myself started to built it with making the notes side-by-side, which took me total 8-9 days.

Loss evaluation: ---------- for GPT Model ----------



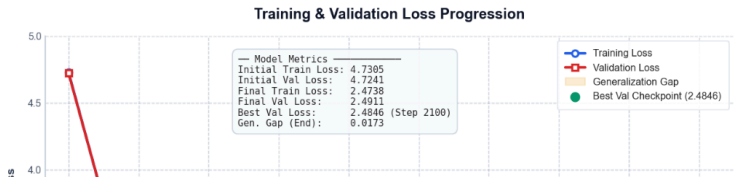
Training data loss	Validation data loss
4.5553	4.5643
1.9163	2.0202
1.5829	1.7487
1.4380	1.6377
1.3479	1.5702
1.2895	1.5341
1.2405	1.5032
1.1960	1.4957
1.1608	1.4957
1.1285	1.4870

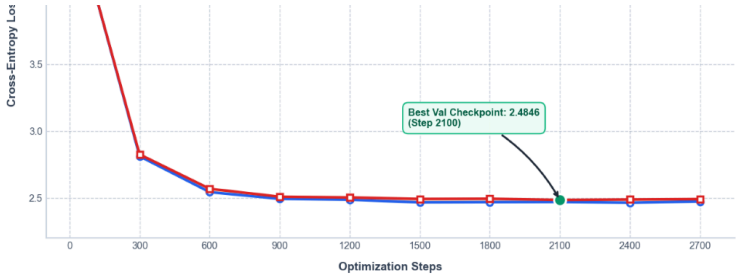
the making of the LLM (Large Language Model) was done in 2 parts : 1. Bigram Model, and 2. GPT Model

Bigram Model

Bigram only included the decoding block for the input given to it, it generates gibblish output as it lacks self-attention, multi-head attention, feedforward block, etc.

Loss evaluation: ---------- for Bigram Model ----------





Training data loss	Validation data loss
4.7305	*4.7241*
2.8110	*2.8249*
2.5434	*2.5682*
2.4932	*2.5088*
2.4863	*2.5035*
2.4665	*2.4921*
2.4683	*2.4936*
2.4696	*2.4846*
2.4638	*2.4879*
2.4738	*2.4911*

Outputs for both the Bigram and the GPT Model

Bigram output:

CEThik brid owindakis b, bth

HAPet bobe d e.

S:

O:3 my d?

LUCous:

Wanthar u qur, t.

War dXENDoate awice my.

Hastarom oroup

Yowhthetof isth ble mil ndill, ath iree sengmin lat Heriliovets, and Win nghir.

Swanousel lind me l.

HAshe ce hiry:

Supr aisspllw y.

Hentofu n Boopetelaves

MPOLI s, d mothakleo Windo whth eisbyo the m dourive we higend t so mower; te

AN ad nterupt f s ar igr t m:

Thin maleronth,

Mad

RD:

WISo myrangoube!

KENob&y, wardsal thes ghesthinin couk ay aney IOUSts I&fr y ce.

J

GPT model output:

hath set muesser, but the contraction

in the priest, and what gaving grows are no more.

GLOUCESTER:

If you dissent was but that the sea,

I doubt, as it arm'd gone: come to come.

BUCKINGHAM:

But you liar was it marry

With Richmondral away: and take it too,

More comes from their whilst I will make

By past incolous was delivering us,

I had you not pay to the live, and fight for the world;

And I'll sing it in error it the dead,

Where I am as she, is a dolemp of prince.

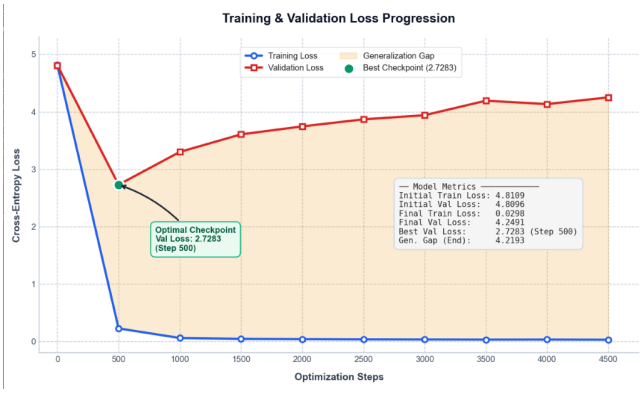
VIRCATIO:

Without be too d

Same GPT tested on TVC rocket gimbal dataset (from my project):

So, here the loss was very low, as the data-set was short upto 3,695 parameters only, whereas, the shakespean text was about 1million parameters.

Loss evaluation: ---------- for test data-set ----------



Training data loss	Validation data loss
4.8109	4.8096
0.2265	2.7283
0.0598	3.2993
0.0434	3.6052
0.0390	3.7437
0.0349	3.8666
0.0334	3.9385
0.0314	4.1912
0.0323	4.1323
0.0298	4.2491

Output:

```
# measurement noise (angle AND angular velocity)
self.R = np.array([[measurement_noise_angle, 0],
                   [0, measurement_noise_ang_velocity]], dtype=float)  # how noisy is the (angle, and angular velocity) reading of the rocket


# observatin matrix — measuring and angle angular velocity directly
self.H = np.array([[1, 0],
                   [0, 1]], dtype=float)  # observation (measurement) matrix


self.I = np.identity(2, dtype=floa
```